Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Department of Computer Science & Engineering

(Artificial Intelligence and Machine Learning)

FORMATIVE ASSESSMENT REPORT

Course: Agentic AI

AutoDiag AI: An Evidence-Linked Multi-Agent System for Automotive Fault Diagnosis

Exact-match retrieval, hybrid RAG, adaptive interviewing and grounded LLM reasoning over OBD-II trouble codes

Submitted by

Sr. no.	Name	PRN
1	Vinayak Prabhakar Pawara	123B1C032
2	Shivam Shivange	123B1C064

Under the guidance of [Prof. Santwana Gudadhe]

Class / Division: [Class – A]

Academic year: 2026–27

Abstract

A modern vehicle reports a fault as a short OBD-II diagnostic trouble code (DTC) such as P0301. The code says what the engine computer detected, not which part failed, so a mechanic or owner still has to look the code up, read the freeze-frame sensor values recorded at the moment of the fault, ask about symptoms, and reason towards the most likely cause. This project builds **AutoDiag AI**, an agentic assistant that does this investigation. Unlike a code-lookup tool or a plain chatbot, it is a **multi-agent pipeline**: a retrieval agent gathers evidence, an interview agent decides whether to pause and ask the user targeted questions, a diagnosis agent reasons only over the gathered evidence, and a response agent turns the reasoning into a structured report.

The pipeline is a LangGraph state machine (retrieval → interview → diagnosis → response) with a conditional human-in-the-loop pause. Retrieval is **exact-match first**: the requested code is looked up directly in a 5,565-record knowledge base built from obdguide.com and a Hugging Face dataset. Only when no exact record exists does the system fall back to **hybrid retrieval** (dense all-MiniLM-L6-v2 vectors in Qdrant plus BM25 keyword search, fused with Reciprocal Rank Fusion). The system, not the model, computes an evidence confidence level, flags unknown codes and safety-critical systems, and attaches the source evidence and a decision trace to every report. The LLM (Groq, openai/gpt-oss-120b) is told to propose only causes that the evidence supports and to decline proprietary codes rather than invent repairs.

The system runs as a Streamlit web application. It is verified by automated regression tests and by scripted evaluations against the real knowledge base: all 1,918 enriched P-codes resolve by exact match, look-alike codes are kept out of the evidence, and unknown or malformed codes produce Low confidence with an explicit warning.

Keywords: agentic AI, multi-agent systems, LangGraph, retrieval-augmented generation, hybrid search, BM25, Reciprocal Rank Fusion, human-in-the-loop, grounding, OBD-II diagnostics.

Contents

Abstract 2
Contents 2
1. Introduction 3
  1.1 Problem 3
  1.2 Why an agent and not a lookup tool or chatbot 3
  1.3 Objectives 3
  1.4 Scope 4
2. Background: agentic AI concepts used 4
  2.1 Multi-agent pipelines and state graphs 4
  2.2 Retrieval-augmented generation and hybrid search 4
  2.3 Grounding, uncertainty and human-in-the-loop 4
3. Requirements 4
4. System architecture 5
5. Agent design 6
  5.1 The agent graph 6
  5.2 Retrieval agent: evidence gathering and self-assessment 7
  5.3 Interview agent: deciding when to ask 7
  5.4 Diagnosis agent and 5.5 Response agent 7
  5.6 Shared state and decision trace 7
6. Prompt engineering 8
  6.1 Key techniques 8
7. Knowledge base and retrieval 8
  7.1 Building the knowledge base 8
  7.2 Retrieval pipeline 9
8. Safety, guardrails and responsible AI 9
  8.1 Mapping to the OWASP Top 10 for LLM applications (2025) 10
9. Division of work between the AI and code 10
10. Implementation 10
11. Testing and evaluation 11
  11.1 Strategy 11
  11.2 Documented test cases 11
  11.3 Performance and coverage 11
12. Example investigation (walkthrough) 12
13. Limitations and future work 12
14. Conclusion 13
References 13
Appendix A: Setup and usage 13
Appendix B: Project facts 14

(Right-click → Update field, or press F9, to refresh the page numbers.)

1. Introduction

1.1 Problem

When the check-engine light comes on, a scan tool returns one or more DTCs, for example `P0301` (cylinder 1 misfire) together with `P0171` (system too lean, bank 1). The same code can have many causes: a misfire may come from a spark plug, an ignition coil, an injector or a vacuum leak. A technician narrows this down by combining three things: what the code means, the **freeze-frame** values stored with it (engine RPM, coolant temperature, fuel trim, manifold pressure), and the symptoms the driver reports. Doing this by hand means searching several websites, mentally cross-checking sensor values and remembering which combinations point to which faults. Owners without that experience often replace the wrong part.

1.2 Why an agent and not a lookup tool or chatbot

A code-lookup website returns a generic list of possible causes and ignores the sensor data. A plain large language model (LLM) can discuss sensor values, but on its own it may confuse similar-looking codes, invent causes for unknown or proprietary codes, and give a confident answer with no visible evidence. The task needs an **agent**: a system that perceives the inputs, **retrieves** authoritative evidence, **decides** whether it knows enough or must **ask** the user, **reasons** over the evidence, and **reports** with its sources and uncertainty visible. Table 1 maps the task onto these agentic capabilities.

Table 1: Agentic capabilities used by AutoDiag AI

Agentic capability	What it means here
Perception	Parses comma-separated DTCs (any case, extra spaces) and optional freeze-frame readings
Knowledge (RAG)	Exact DTC lookup in a 5,565-record knowledge base; hybrid dense + BM25 fallback
Decision / planning	Conditional graph route: ask follow-up questions, or proceed to diagnosis
Interaction	Targeted questions with a stated reason; the user's answers re-enter the pipeline
Reasoning	LLM ranks root causes using evidence, freeze-frame rules and the user's answers
Self-assessment	Code computes High / Medium / Low evidence confidence and raises warnings
Transparency	Every report carries its evidence records and a step-by-step decision trace
Guardrails	Code format validation, unknown-code and safety warnings, no-fabrication prompt rule

1.3 Objectives

1. Build a multi-agent LLM pipeline that turns DTCs and freeze-frame data into a structured diagnostic report.
2. Ground every diagnosis in retrieved knowledge-base evidence, with exact code matches taking priority over fuzzy search.
3. Let the agent decide when information is missing and ask the user focused, explainable questions.
4. Make uncertainty and safety risks explicit instead of hiding them behind a confident answer.
5. Verify retrieval, interview and confidence behaviour with reproducible tests that do not depend on the LLM.

1.4 Scope

In scope: the four agents, the knowledge-base build (scraping, merging, embedding), exact and hybrid retrieval, the interview loop, prompt design, confidence and safety handling, and the Streamlit interface. Out of scope: reading codes directly from a vehicle (codes are typed in), make- and model-specific service manuals, live sensor streams, and any repair action. The output is guidance for a qualified technician, not a confirmed diagnosis.

2. Background: agentic AI concepts used

2.1 Multi-agent pipelines and state graphs

An LLM agent wraps a model in a loop of observing, deciding and acting. Complex tasks are often split across several specialised agents, each with a narrow role and its own prompt, that pass a shared state between them. LangGraph models such a system as an explicit **state graph**: nodes are agents, edges (including conditional edges) define the control flow, and a typed state object carries data between nodes. This makes the flow inspectable and testable, and lets the graph stop early and resume later. AutoDiag AI uses this to pause for user input.

2.2 Retrieval-augmented generation and hybrid search

Retrieval-augmented generation (RAG) (Lewis et al., 2020) retrieves relevant documents and gives them to the model, so answers come from data rather than from the model's memory. Dense retrieval embeds text into vectors and finds semantically similar records. It handles paraphrases but can rank a look-alike code such as `P30D8` close to `P0300`. Keyword retrieval with BM25 (Robertson & Zaragoza, 2009) rewards exact term overlap. **Reciprocal Rank Fusion** (Cormack et al., 2009) merges the ranked lists with $\Sigma\ 1/(k + \text{rank})$, without calibrating their different score scales.

2.3 Grounding, uncertainty and human-in-the-loop

LLMs can hallucinate plausible but wrong facts, which is dangerous when the advice concerns brakes, airbags or high-voltage systems. Accepted mitigations are to restrict the model to supplied evidence, to compute trust signals (such as confidence) in code rather than asking the model, to show sources, and to keep a human in the loop when the system lacks information. The OWASP Top 10 for LLM Applications (2025) lists misinformation, excessive agency and improper output handling among the main risks. Section 8 maps the design to it.

3. Requirements

Table 2: Functional requirements

#	Functional requirement
F1	Accept one or more DTCs (comma separated, any case) and optional freeze-frame values: RPM, coolant temperature, fuel trim, MAP
F2	Retrieve the exact knowledge-base record for each code; fall back to hybrid search only when none exists
F3	Ask targeted follow-up questions when freeze-frame or symptom context is missing, and resume with the answers
F4	Produce a report: fault summary, ranked root causes, severity with explanation, diagnostic steps, repair estimate, immediate action
F5	Show the evidence records, the evidence confidence, warnings and the decision trace with every report
F6	Flag unsupported code formats, codes with no knowledge-base match, and safety-critical systems

Table 3: Non-functional requirements

#	Non-functional requirement
N1	Grounding: causes and steps must come from retrieved evidence or user input; no fabricated repairs for unknown codes
N2	Precision: an exact code match must never be displaced by a semantically similar but different code
N3	Transparency: confidence and warnings are system-generated, not LLM claims
N4	Low cost: free-tier Groq inference, open-source local embeddings, a hosted Qdrant Cloud cluster
N5	Testability: retrieval and interview logic testable without network access or a real LLM
N6	Security: API keys only in an untracked .env file

4. System architecture

Figure 1 shows the complete system. The user works in a Streamlit web page. The page invokes a compiled LangGraph pipeline of four agents that share one state object. The retrieval agent reads a local copy of the knowledge base (for exact matches and BM25) and, only when needed, the Qdrant Cloud vector collection. The diagnosis and response agents call the Groq API. The knowledge base itself is built offline, once, by a scrape → merge → ingest pipeline.

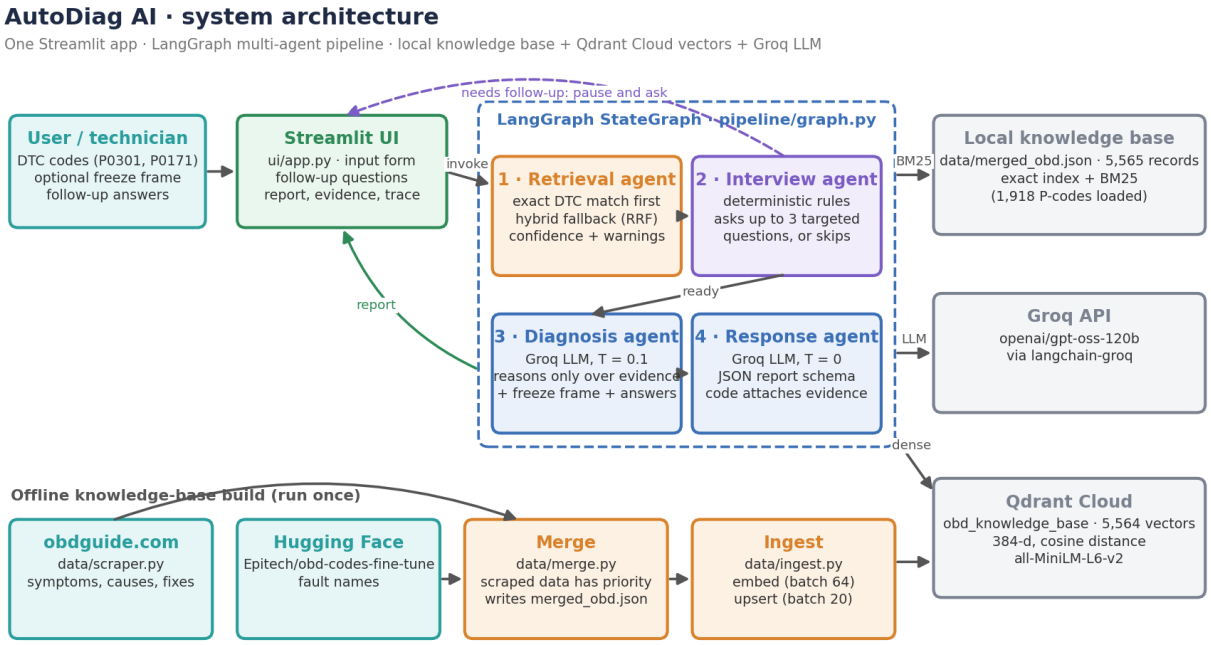


Figure 1: System architecture (user → Streamlit → LangGraph agents → knowledge base, vector store and LLM)

Table 4: Technology stack

Layer	Technology	Role in the agent
Workflow	LangGraph StateGraph, LangChain core	Agent nodes, conditional routing, shared typed state
LLM	Groq · openai/gpt-oss-120b (langchain-groq)	Diagnosis reasoning (T = 0.1) and JSON report formatting (T = 0)
Embeddings	all-MiniLM-L6-v2 (Sentence Transformers), 384-d	Semantic vectors for fallback retrieval
Vector store	Qdrant Cloud, cosine distance	Dense nearest-neighbour search over 5,564 records
Keyword search	rank-bm25 (BM25Okapi)	Exact-code index and keyword ranking (local, in memory)
Data build	requests + BeautifulSoup, Hugging Face datasets	Scrape obdguide.com, merge with Epitech/obd-codes-fine-tune
Interface	Streamlit	Input form, follow-up questions, report, evidence and trace views
Language / config	Python 3.11+, python-dotenv	Application code; credentials from .env

5. Agent design

5.1 The agent graph

The pipeline is an explicit LangGraph state machine with four nodes (Figure 2). Every investigation enters the **retrieval** node, then the **interview** node. A conditional edge reads `needs_follow_up`. If it is set, the graph ends early and the questions are shown to the user. When the user continues, the graph is invoked again with

`symptom_answers` and `interview_complete`, and this time the route goes to **diagnosis** and **response**. The graph has no cycles, so each invocation runs at most four nodes and makes at most two LLM calls.

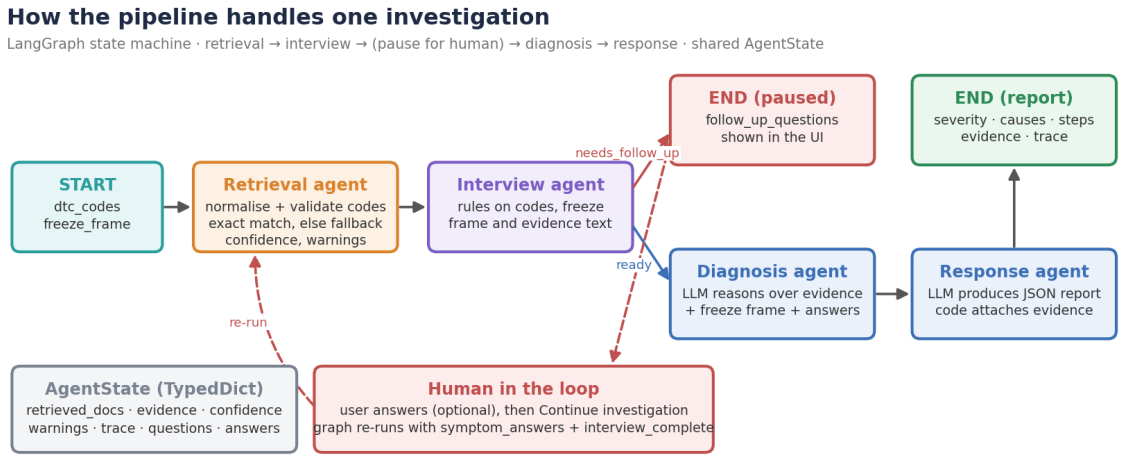


Figure 2: The agent graph: retrieval → interview → (pause for the user) → diagnosis → response

```
graph = StateGraph(AgentState)
graph.add_node("retrieval", retrieval_agent.run)
graph.add_node("interview", interview_agent.run)
graph.add_node("diagnosis", diagnosis_agent.run)
graph.add_node("response", response_agent.run)
graph.set_entry_point("retrieval")
graph.add_edge("retrieval", "interview")
graph.add_conditional_edges("interview",
    lambda s: "follow_up" if s.get("needs_follow_up") else "diagnosis",
    {"follow_up": END, "diagnosis": "diagnosis"})     # human-in-the-loop pause
graph.add_edge("diagnosis", "response")
graph.add_edge("response", END)
```

Listing 1: The agent graph (pipeline/graph.py, simplified)

5.2 Retrieval agent: evidence gathering and self-assessment

The retrieval agent (`agents/retrieval_agent.py`) normalises each code (strip, upper-case) and validates it against the OBD-II pattern `[PCBU][0-9A-F]{4}`. It then performs an **exact lookup** for every code. Only if no code matched at all does it run `hybrid_search` on a query built from the codes and freeze-frame terms. This ordering is the key design decision. In an earlier version, fuzzy search for `P0300` also returned `P30D8` and `P30DD`, unrelated codes that look similar to an embedding model, and these leaked into the diagnosis. The agent then assesses its own evidence in code:

- **Confidence:** High when every code matched exactly, Medium when some evidence was found, Low when none was.
- **Unknown-code warning:** every code without an exact match is named, for example “No direct OBD-II knowledge-base match for: P9999.”
- **Safety warning:** if the evidence mentions high voltage, electric shock, airbags, brakes, fuel leaks or overheating, the report warns against DIY repair.
- **Evidence for the UI:** a compact `retrieved_evidence` list (code, fault name, source, symptoms, causes, repair steps).

5.3 Interview agent: deciding when to ask

The interview agent (`agents/interview_agent.py`) decides whether the investigation has enough context. Its questions are generated by **deterministic rules**, not by the LLM, so each one has a fixed, explainable reason that is shown to the user as help text (Table 5). If answers are already present, it marks the interview complete and lets the graph continue. Answers are optional, so the user can skip and still get a report.

Table 5: Interview rules

Trigger	Question asked	Reason shown to the user
No freeze-frame data	When does the fault occur: at idle, accelerating, under load, or always?	Separates intermittent, load-related and idle-related faults
Any P03xx (misfire) code	Does the engine shake at idle, only when accelerating, or continuously?	Narrows ignition vs fuel vs air-intake causes
P0171 / P0174, or "lean" in evidence	Hissing sound, rough idle, or reduced fuel economy?	Assesses a possible vacuum or unmetered-air leak

5.4 Diagnosis agent and 5.5 Response agent

The **diagnosis agent** builds one prompt from the codes, the freeze-frame values, the user's answers, the system-computed confidence and warnings, and the retrieved records (symptoms, causes, repair steps), and asks the Groq model (temperature 0.1) for a structured analysis. The **response agent** asks the model (temperature 0) to convert that analysis into a fixed JSON schema, strips any Markdown fences and parses it. If parsing fails it keeps the raw text instead of crashing. It then overwrites three fields with **system-generated metadata**: `evidence`, `confidence` and `warnings`. These fields are never produced by the LLM, so the model cannot raise its own confidence or drop a safety warning.

5.6 Shared state and decision trace

All agents read and write one `AgentState` TypedDict: inputs (`dtc_codes`, `freeze_frame`), evidence (`retrieved_docs`, `retrieved_evidence`, `confidence`, `warnings`), interview data (`follow_up_questions`, `symptom_answers`, `interview_complete`, `needs_follow_up`) and outputs (`diagnosis`, `response`). Each agent appends a short, user-safe event to `trace`, for example "Exact DTC evidence was available, so unrelated fuzzy matches were excluded." The UI shows this as the **diagnostic decision trace**, so the user can see how the report was produced.

6. Prompt engineering

The diagnosis prompt has two parts: a fixed system prompt that defines the role, the domain rules and a hard refusal rule, and a user prompt assembled at run time from the state. Table 7 lists the techniques used.

Table 6: Structure of the diagnosis prompt

Section	Technique	Purpose
Role	Role prompting	Expert automotive diagnostic engineer with OBD-II knowledge
Task steps	Step-by-step instructions	Analyse codes → use freeze frame → reason about root causes → structured diagnosis
Freeze-frame rules	Encoded domain heuristics	Fuel trim > +10 % = lean; < −10 % = rich; MAP < 40 kPa / > 80 kPa at idle; high RPM = load; low coolant = cold start
Critical rule	Explicit refusal template	Non-standard or unknown codes get a fixed "cannot diagnose" answer, never invented causes
Evidence block	Retrieval grounding	Retrieved symptoms, causes and repair steps per code, marked as the knowledge base
Trust signals	Dynamic context	System confidence and warnings injected; the model must state uncertainty when confidence is Low
User answers	Interactive context	Follow-up answers are passed in as evidence
Output format	Structured output	Five numbered parts; the response agent then enforces an exact JSON schema

6.1 Key techniques

- **Grounding by instruction.** "Only propose causes supported by the retrieved knowledge base or supplied inputs." Combined with exact-match retrieval, the model sees only the right code's evidence.
- **Refusal for the unknown.** A fixed answer template for proprietary codes stops the model from inventing a plausible-sounding repair.

- **Domain heuristics in the prompt.** Freeze-frame thresholds give the model the rules an experienced technician uses, so it ties the cause to the sensor data.
- **Two-stage generation.** Reasoning (T = 0.1) and formatting (T = 0) are separate calls. Strict JSON output does not constrain the reasoning, and a formatting failure does not lose the diagnosis.
- **Code-owned trust fields.** Confidence, warnings and evidence are attached after generation, so prompt tricks cannot change them.

```
CRITICAL RULE: If the DTC code is not a standard OBD-II code or if the retrieved
knowledge base has no relevant information, you MUST respond with:
- fault_summary: "Non-standard or proprietary DTC code - not in OBD-II knowledge base"
- root_causes: ["Cannot diagnose - proprietary code requires manufacturer-specific documentation"]
- severity: "Unknown"
Do NOT fabricate causes or repair steps for unknown codes.

Only propose causes supported by the retrieved knowledge base or supplied inputs. If confidence
is Low or warnings show no direct DTC match, clearly state the uncertainty and request
manufacturer documentation or qualified inspection. Preserve any safety warning in the
immediate action.
```

Listing 2: Part of the diagnosis prompts (agents/diagnosis_agent.py)

7. Knowledge base and retrieval

7.1 Building the knowledge base

`data/scraper.py` collects the code list from obdguide.com and scrapes each code page for its fault name, symptoms, causes and fixes (0.5 s delay between requests). `data/fix_missing.py` re-scrapes records that came back empty. `data/merge.py` merges these records with the Hugging Face dataset Epitech/obd-codes-fine-tune. Scraped data has priority, and the dataset fills in missing fault names. The merge also builds one `text` field per code for embedding. `data/ingest.py` embeds every enriched record with all-MiniLM-L6-v2 and uploads it to the Qdrant collection `obd_knowledge_base` (384-d, cosine).

Table 7: Knowledge-base contents (data/merged_obd.json)

Item	Value
Total DTC records	5,565 (powertrain P 1,918 · body B 1,852 · chassis C 981 · network U 814)
Records with symptoms or causes	5,564, all embedded in Qdrant
Records with repair steps	5,561
Local exact / BM25 index	1,918 enriched P-codes, loaded in memory at start-up

7.2 Retrieval pipeline

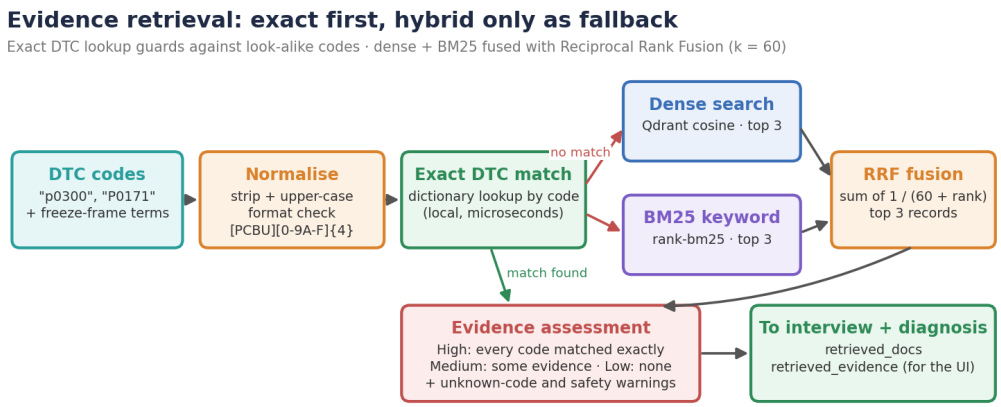


Figure 3: Evidence retrieval: exact first, hybrid fallback with RRF

- **Exact match:** a dictionary from code to records gives a direct lookup. Exact records carry `source = exact_dtc`.
- **Dense search (fallback):** the query (codes + "RPM 2500 fuel trim +15%" style terms) is embedded and the top results come from Qdrant by cosine similarity.

- **BM25 (fallback):** keyword ranking over the same record text, which catches literal code and term overlap.
- **Fusion:** each list contributes 1/(60 + rank) per code and the top 3 fused records become the evidence.

8. Safety, guardrails and responsible AI

Wrong automotive advice can cost money or cause injury, so safety is built in layers, and the layers that matter most are written in ordinary code, not left to the model (Table 9).

Table 8: Layered guardrails

Layer	Mechanism	Risk addressed
1. Input validation	Codes normalised and checked against [PCBU][0-9A-F]{4}; empty input rejected in the UI	Malformed input, wasted LLM calls
2. Exact-first retrieval	Fuzzy search runs only when no exact record exists	Look-alike codes polluting evidence
3. Grounding rules	Evidence-only instruction and refusal template for unknown codes	Hallucinated causes and repairs
4. Code-owned trust signals	Confidence, warnings and evidence attached by code after generation	Over-confident or altered reports
5. Safety warnings	Keyword scan of evidence for high-voltage, airbag, brake, fuel, overheating risks	Unsafe DIY repairs
6. Human in the loop	Interview pause; report framed as guidance for a qualified technician	Excessive autonomy
7. Robust parsing	JSON parse failure falls back to raw text; UI uses .get() defaults	Crashes on malformed model output
8. Secret handling	API keys only in .env (git-ignored); .env.example has placeholders	Credential leaks

8.1 Mapping to the OWASP Top 10 for LLM applications (2025)

Table 9: OWASP LLM Top 10 coverage

OWASP risk	How AutoDiag AI addresses it	Status
LLM01 Prompt injection	Codes are format-checked; free-text answers reach the prompt without a dedicated guard	Partial
LLM05 Improper output handling	JSON parsed defensively; Streamlit renders text, not raw HTML	Covered
LLM06 Excessive agency	The agents have no tools that act on the world; output is advice only	Covered
LLM08 Vector / embedding weaknesses	Exact match bypasses vector search whenever possible	Covered
LLM09 Misinformation	Evidence-only rule, refusal template, confidence and warnings from code	Covered
LLM10 Unbounded consumption	Acyclic graph: at most two LLM calls per investigation	Covered

The free-text follow-up answers are the remaining injection surface. They are short and only change the wording of the diagnosis, because confidence, warnings and evidence are written by code afterwards. An input guard for these answers is listed as future work.

9. Division of work between the AI and code

A recurring design principle is to let the model do what it is good at, and keep everything that must be exact, explainable or safe in ordinary code (Table 11).

Table 10: Responsibilities

The LLM decides	Code decides
How the codes, sensor values and symptoms combine into a probable root cause	Which knowledge-base records are evidence (exact match first)

The LLM decides	Code decides
The order of diagnostic steps by likelihood	Whether a code is valid and whether it exists in the knowledge base
The severity level and its explanation	The evidence confidence level (High / Medium / Low)
Wording of the summary, immediate action and repair estimate	Safety and unknown-code warnings
—	Whether to ask follow-up questions, and which ones
—	The report schema, evidence attachment and decision trace

10. Implementation

Table 11: Code organisation (about 1,140 lines of Python in 22 files)

Area	Main modules
Agents	agents/retrieval_agent.py (AgentState, evidence, confidence), interview_agent.py, diagnosis_agent.py, response_agent.py
Orchestration	pipeline/graph.py (LangGraph StateGraph with conditional interview route)
Retrieval	retrieval/hybrid_retriever.py (Qdrant + RRF), bm25_retriever.py (BM25 + exact index), embedder.py
Data pipeline	data/scraper.py, fix_missing.py, merge.py, ingest.py; data/merged_obd.json (knowledge base)
Interface	ui/app.py (Streamlit, session state for the interview pause), run.py (launcher)
Tests	tests/test_investigation_flow.py (unittest with a stub retriever); tests/explore.py (manual search probe)

The Streamlit page caches the compiled graph with `st.cache_resource`, so models and indexes load once per server. The interview pause is implemented with `st.session_state`: the first `invoke` returns the questions, and **Continue investigation** re-invokes the graph with the answers, then reruns the page to show the report. The app runs locally with `python -m streamlit run ui/app.py` once `.env` holds the Groq and Qdrant credentials and the collection has been populated with `python -m data.ingest`.

11. Testing and evaluation

11.1 Strategy

LLM output is non-deterministic, so the evaluation separates two questions. (1) **Is the agent machinery correct?** Retrieval, confidence, warnings and interview routing are deterministic code, so they are tested exactly: by automated unit tests with a stub retriever, and by scripted runs of the real retrieval and interview agents against the real knowledge base (local exact/BM25 index, no network). (2) **Does the real model reason well?** This is checked by end-to-end runs in the Streamlit app with real Groq calls (Section 12).

Table 12: Automated test suite (unittest, tests/test_investigation_flow.py)

Test	What is verified	Result
test_exact_dtc_match_excludes_fuzzy_neighbours	P0300 returns only P0300; fallback (which would return P30D8, P30DD) is never called; confidence High	Pass
test_unknown_code_has_warning_and_low_confidence	P9999 gives Low confidence and a “No direct … match” warning	Pass
test_interview_requests_questions_then_accepts_answers	P0301 without context asks questions; with answers the interview completes	Pass
Total		3 / 3 passed

11.2 Documented test cases

Table 13: Test cases (TC1–TC3 automated; TC4–TC10 scripted against the real knowledge base on 22 Oct 2026)

ID	Objective	Input	Expected result	Result
TC1	Exact match beats look-alikes	P0300 (stub fallback offers P30D8, P30DD)	Evidence = [P0300]; no fallback call; High	Pass
TC2	Unknown code handled honestly	P9999	No evidence; Low; warning names P9999	Pass
TC3	Interview pause and resume	P0301, no freeze frame → then answers	Questions requested → then interview complete	Pass
TC4	Multi-fault exact retrieval	P0301, P0171	Both records; High; no fallback; no warnings	Pass
TC5	Input normalisation	"p0420", " P0430 "	Normalised to P0420, P0430; both matched; High	Pass
TC6	Safety warning	P0217 (engine overheat)	Exact match plus safety-critical warning	Pass
TC7	Malformed code	X123	"Unsupported DTC format" and no-match warnings; Low	Pass
TC8	Partial match	P0300, P9999	P0300 evidence; Medium; warning for P9999 only	Pass
TC9	Interview adapts to context	P0301, P0171 with / without freeze frame	3 questions without; 2 with (operating-condition question dropped)	Pass
TC10	Exact-match coverage	Every enriched P-code in the knowledge base	1,918 / 1,918 resolve by exact match	Pass

11.3 Performance and coverage

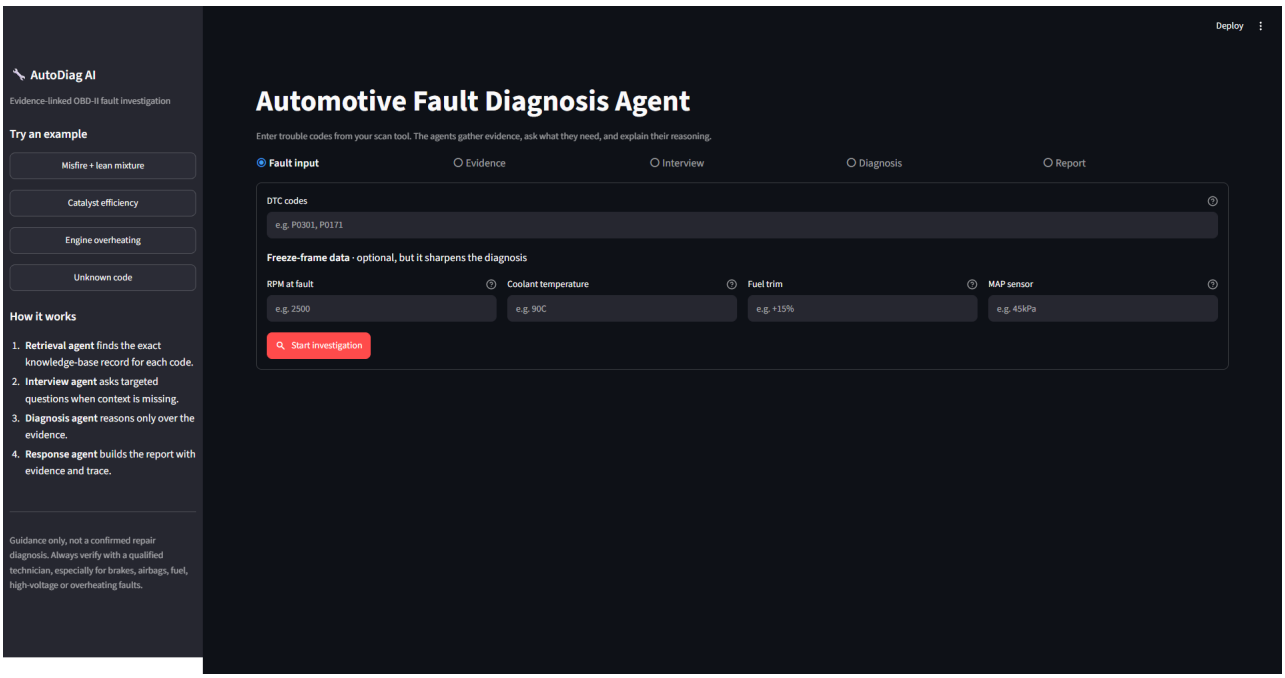
On a laptop, exact lookup of all 5,564 enriched codes took about 6 ms in total, and one BM25 query over the 1,918-record index took about 1.6 ms. Retrieval cost is negligible next to the two Groq calls, which dominate the end-to-end time; LLM latency was not benchmarked. The coverage run also exposed a gap. The local exact index loads only P-codes, so B, C and U codes (3,646 records that do exist in Qdrant) never match exactly. They go to the hybrid fallback and are reported with a "No direct match" warning and Low or Medium confidence, which is conservative but understates the evidence. Extending the exact index to all prefixes is the first item in Section 13.

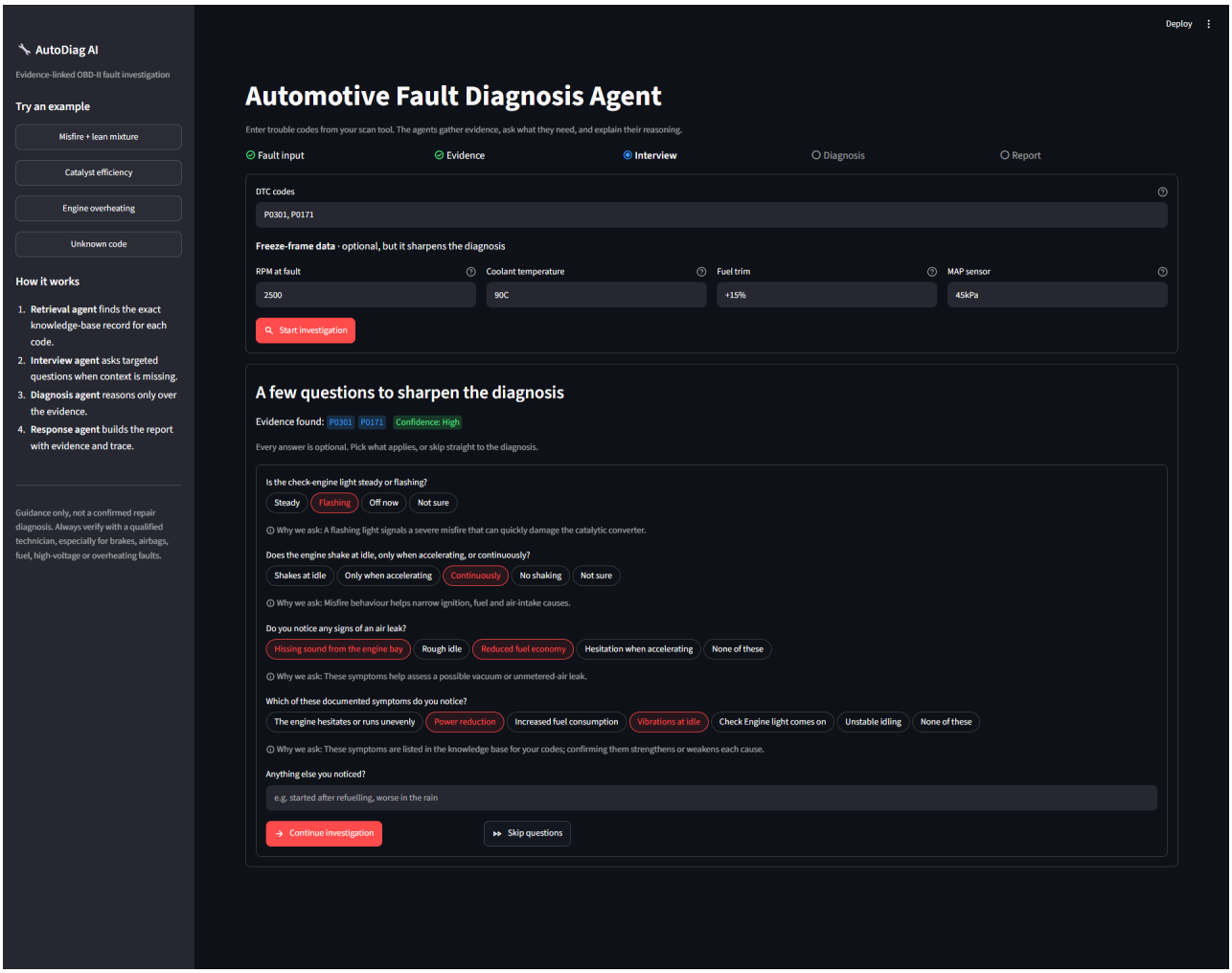
12. Example investigation (walkthrough)

Inputs `P0301, P0171` with no freeze frame. The first five steps are the real trace produced by the agents. The diagnosis step is illustrative, and Figure 4 shows an actual report from the app.

Table 14: Walkthrough of one investigation

Step	What happens [agent / trace event]
User	Enters P0301, P0171 and clicks Start investigation
Retrieval	[exact_dtc_match] Found direct knowledge-base matches for: P0301, P0171
Retrieval	[fallback_retrieval] Exact DTC evidence was available, so unrelated fuzzy matches were excluded
Retrieval	[evidence_assessment] Prepared 2 evidence record(s); confidence is High
Interview	[interview] Requested 3 targeted follow-up question(s): operating condition, misfire behaviour, air-leak signs → graph pauses
User	Answers "shakes at idle" and "slight hissing near intake", clicks Continue investigation
Diagnosis	[diagnosis] Ranks causes from the P0301 and P0171 evidence plus the answers, e.g. a vacuum leak causing a lean misfire
Response	[report_generation] JSON report with severity, causes and steps, plus evidence, High confidence and the trace


Deploy
AutoDiag AI
Evidence-linked OBD-II fault investigation
Try an example
Misfire + lean mixture
Catalyst efficiency
Engine overheating
Unknown code
How it works
1. Retrieval agent finds the exact knowledge-base record for each code.
2. Interview agent asks targeted questions when context is missing.
3. Diagnosis agent reasons only over the evidence.
4. Response agent builds the report with evidence and trace.
Guidance only, not a confirmed repair diagnosis. Always verify with a qualified technician, especially for brakes, airbags, fuel, high-voltage or overheating faults.
Automotive Fault Diagnosis Agent
Enter trouble codes from your scan tool. The agents gather evidence, ask what they need, and explain their reasoning.
Fault input
Evidence
Interview
Diagnosis
Report
DTC codes
e.g. P0301, P0171
Freeze-frame data · optional, but it sharpens the diagnosis
RPM at fault
e.g. 2500
Coolant temperature
e.g. 90C
Fuel trim
e.g. +15%
MAP sensor
e.g. 45kPa
Start investigation


Deploy
AutoDiag AI
Evidence-linked OBD-II fault investigation
Try an example
Misfire + lean mixture
Catalyst efficiency
Engine overheating
Unknown code
How it works
1. Retrieval agent finds the exact knowledge-base record for each code.
2. Interview agent asks targeted questions when context is missing.
3. Diagnosis agent reasons only over the evidence.
4. Response agent builds the report with evidence and trace.
Guidance only, not a confirmed repair diagnosis. Always verify with a qualified technician, especially for brakes, airbags, fuel, high-voltage or overheating faults.
Automotive Fault Diagnosis Agent
Enter trouble codes from your scan tool. The agents gather evidence, ask what they need, and explain their reasoning.
Fault input
Evidence
Interview
Diagnosis
Report
DTC codes
P0301, P0171
Freeze-frame data · optional, but it sharpens the diagnosis
RPM at fault
2500
Coolant temperature
90C
Fuel trim
+15%
MAP sensor
45kPa
Start investigation
A few questions to sharpen the diagnosis
Evidence found: P0301 P0171 Confidence: High
Every answer is optional. Pick what applies, or skip straight to the diagnosis.
Is the check-engine light steady or flashing?
Steady
Flashing
Off now
Not sure
Why we ask: A flashing light signals a severe misfire that can quickly damage the catalytic converter.
Does the engine shake at idle, only when accelerating, or continuously?
Shakes at idle
Only when accelerating
Continuously
No shaking
Not sure
Why we ask: Misfire behaviour helps narrow ignition, fuel and air-intake causes.
Do you notice any signs of an air leak?
Hissing sound from the engine bay
Rough idle
Reduced fuel economy
Hesitation when accelerating
None of these
Why we ask: These symptoms help assess a possible vacuum or unmetered-air leak.
Which of these documented symptoms do you notice?
The engine hesitates or runs unevenly
Power reduction
Increased fuel consumption
Vibrations at idle
Check Engine light comes on
Unstable idling
None of these
Why we ask: These symptoms are listed in the knowledge base for your codes; confirming them strengthens or weakens each cause.
Anything else you noticed?
e.g. started after refuelling, worse in the rain
Continue investigation
Skip questions

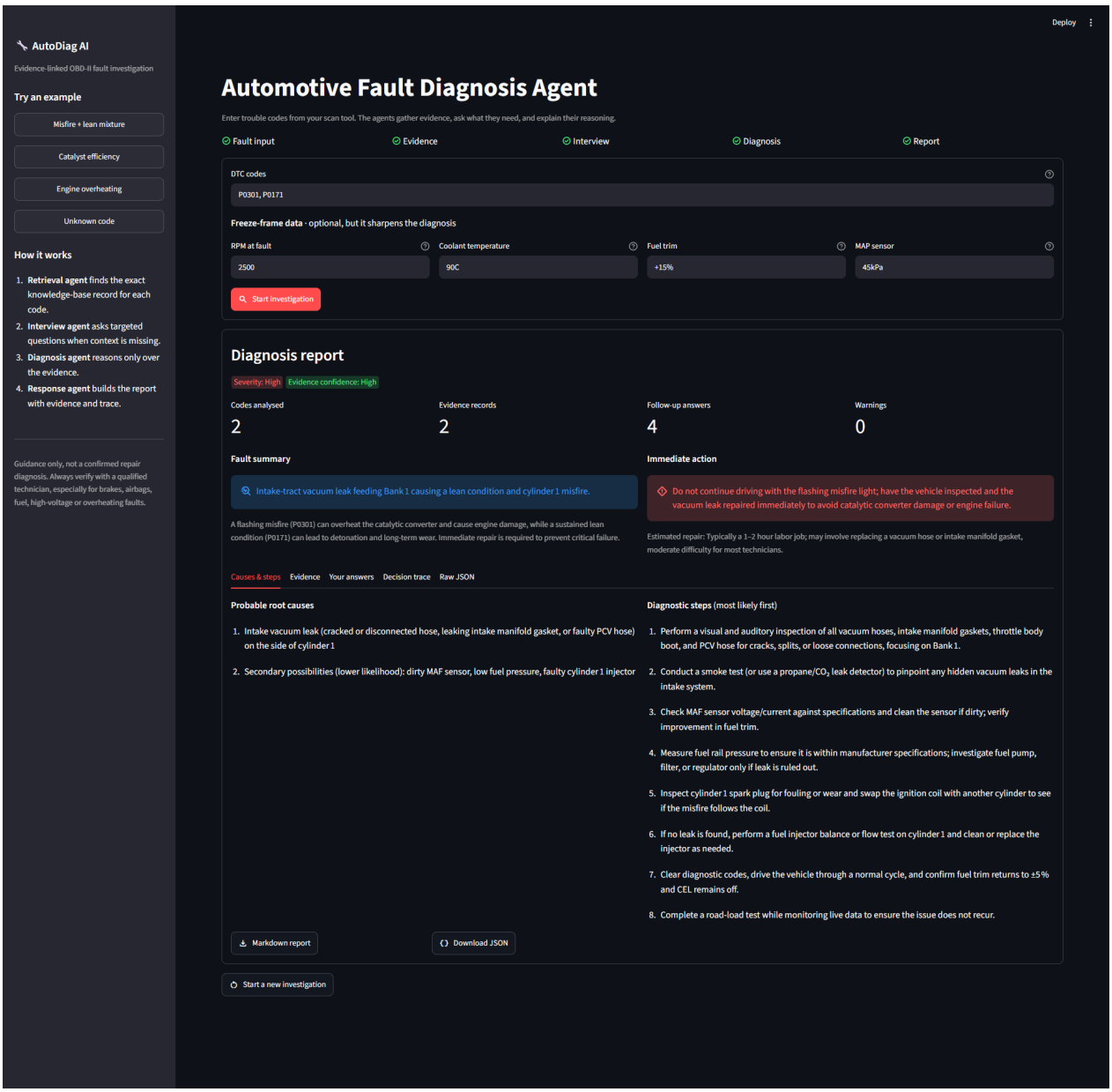


Figure 4: Streamlit app: P0301 with RPM 2500, coolant 92 °C, fuel trim +18 %, MAP 47 kPa → "Cylinder 1 misfire due to probable vacuum leak" (docs/demo_input.png, docs/demo_output.png; captured before the interview step was added)

The positive fuel trim (+18 %) is the deciding clue. The prompt's rule "fuel trim > +10 % suggests a lean condition" leads the model to rank a vacuum leak above the ignition causes, and the first diagnostic step it proposes is to inspect the intake manifold for leaks.

13. Limitations and future work

- **Exact index covers P-codes only.** B, C and U codes fall back to fuzzy search (Section 11.3). Indexing all prefixes for exact match is a one-line filter change.
- **Fallback only when nothing matches.** For a mix of known and unknown codes (P0300 + P9999), the unknown code gets no contextual retrieval; per-code fallback would help.
- **Generic knowledge.** Records are generic OBD-II data, not make- or model-specific service information or technical service bulletins.
- **Rule-based interview.** Three fixed rules; LLM-generated questions checked against the evidence could cover more fault families.

- **Model evaluation.** No automated scoring of diagnosis quality yet; a labelled case set with an LLM-as-judge or technician review is the next step.
- **Input guard and observability.** Add an injection filter for free-text answers, and record per-node latency and token use.
- **Vehicle connection.** Reading codes and live data from an ELM327 OBD-II adapter would remove manual entry.

14. Conclusion

AutoDiag AI shows that an LLM becomes a dependable diagnostic assistant when it is part of a controlled multi-agent system rather than a free-running chatbot. A retrieval agent grounds each case in exact knowledge-base evidence and turns to hybrid RAG only when it must. An interview agent knows when to stop and ask the user. A diagnosis agent reasons over evidence and sensor data under explicit no-fabrication rules, and a response agent delivers a structured report. Code, not the model, owns the trust signals: confidence, unknown-code and safety warnings, the evidence and the decision trace. Deterministic tests confirm that exact matches cannot be displaced by look-alike codes and that unknown codes are reported honestly, and they also exposed the P-code-only exact index as the clearest next improvement.

References

[1] Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.

[2] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

[3] Robertson, S., Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in IR 3(4).

[4] Cormack, G. V., Clarke, C. L. A., Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.

[5] Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP 2019.

[6] Wang, W. et al. (2020). MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. NeurIPS 2020.

[7] OWASP Foundation (2025). OWASP Top 10 for Large Language Model Applications.

[8] LangChain (2026). LangGraph documentation: StateGraph and conditional edges. https://langchain-ai.github.io/langgraph/

[9] Qdrant (2026). Qdrant vector database documentation. https://qdrant.tech/documentation/

[10] SAE International (2016). J2012: Diagnostic Trouble Code Definitions.

Appendix A: Setup and usage

Table 15: Commands

Command	What it does
python -m pip install -r requirements.txt	Install dependencies into the virtual environment
Copy-Item .env.example .env	Create the credentials file (QDRANT_URL, QDRANT_API_KEY, GROQ_API_KEY)
python -m data.ingest	Embed the knowledge base and upload it to Qdrant (once per cluster)
python -m streamlit run ui/app.py	Start the app at http://localhost:8501
python -m unittest discover -s tests -p "test_*.py" -v	Run the regression tests

Appendix B: Project facts

Table 16: Summary

Item	Value
Application code	22 Python files, about 1,140 lines
Agents	4 (retrieval, interview, diagnosis, response) in one LangGraph StateGraph
LLM	Groq openai/gpt-oss-120b; at most 2 calls per investigation
Knowledge base	5,565 DTC records (P/B/C/U); 5,564 embedded; 1,918 P-codes in the exact/BM25 index
Embedding model	all-MiniLM-L6-v2, 384 dimensions, cosine distance in Qdrant Cloud
Retrieval	Exact DTC match first; fallback dense + BM25 fused with RRF (k = 60), top 3
Tests	3 automated unit tests (3 / 3 pass) + 7 scripted knowledge-base checks
Interface	Streamlit web app with interview pause, evidence and decision-trace views